Birla Institute of Technology & Science, Pilani
Work Integrated Learning Programmes Division
Second Semester 2024-2025

Comprehensive Examination
(EC-3 Regular)

Course No.	: AIMLCZG511
Course Title	: Deep Neural Networks
Nature of Exam	: Open Book
Weightage	: 40%
Duration	: 2 Hours
Date of Exam	: 07-09-2025

No. of Pages = 03
No. of Questions = 06

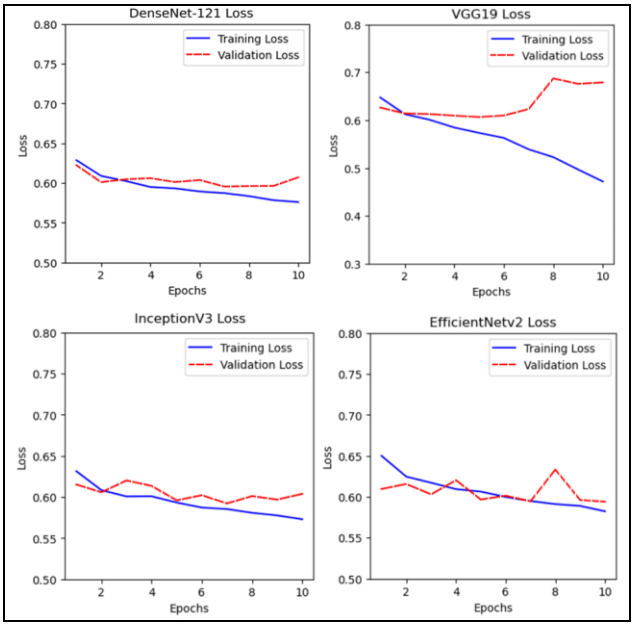
Note to Students:
1. Please follow all the *Instructions to Candidates* given on the cover page of the answer book.
2. All parts of a question should be answered consecutively. Each answer should start from a fresh page.
3. Assumptions made if any, should be stated clearly at the beginning of your answer.

Q.1. Answer the following questions: (limit answers to 1-2 sentences) **[5 Marks]**

A. Suppose you are training neural networks on 100x100 images to predict 5 classes. Neural network A consists of a single linear layer followed by a softmax output activation. Neural network B consists of a sequence of layers with dimensions 128, 512, and 32, respectively, followed by a softmax output activation. Assuming that both neural networks are trained using an identical procedure (e.g. batch size, learning rate, epochs, etc), and neither contains hidden activations, what can you generally expect about the relative performance of A and B on the test data? (2M)

B. Suppose you set up and train a neural network on a classification task and converge to a final loss value. Keeping everything in the training process the same (e.g. learning rate, optimizer, epochs). It is possible to reach a lower loss value by ONLY changing the network initialization. (2M)

C. Four deep neural network models are trained on a classification task, and below are the plots of their losses. Based on these plots, which model is overfitting? (1M)



Answer Key

A. Neural network A will likely underperform compared to B because A has very limited capacity (only linear mapping), while B has more parameters and representational power, though it may risk overfitting.
B. Yes, different random initializations can lead to different convergence behaviors, and sometimes a better (lower) final loss can be reached solely by changing initialization.
C. VGG19 is overfitting since its validation loss is consistently higher and diverges from training loss, while training loss continues to decrease.

Q.2. Answer the following **[5 Marks]**

A. Assume a company asks you to develop an application that can predict which kind of bird is depicted in a given image. Which kind of task is this? List and explain the individual steps you would follow to solve this problem using deep learning. (end-to-end steps get full marks) (2M)

B. Draw the computational graph corresponding to the following equations:

$$o = a(\mathbf{w} \cdot \mathbf{x} + b)$$

$$o = f(x) = f\Big(g_1(h_1(x), h_2(x)),\ g_2(h_2(x), h_3(x))\Big)$$

Clearly show the flow of inputs, intermediate nodes (e.g., weighted sum, hidden functions), and final output. (3M)

Answer Key

A. Task type: Supervised image classification (multi-class).

1. Acquire a suitably large, labeled training set of the relevant kinds of birds
2. Use transfer learning to re-use existing models' capabilities with regards to general feature extraction. This will significantly cut down on training time and the required training set size.
3. Optimally select a pre-trained model which was trained on a related domain (birds, nature, small animals, etc.). The closer the better**.**
4. Cut off the last layers of the pre-trained network and replace them with fully connected layers. These layers are the only ones which can be changed during training.
5. If this does not work I would try to remove the training restriction on the feature extraction layers to allow fine tuning of the whole model.

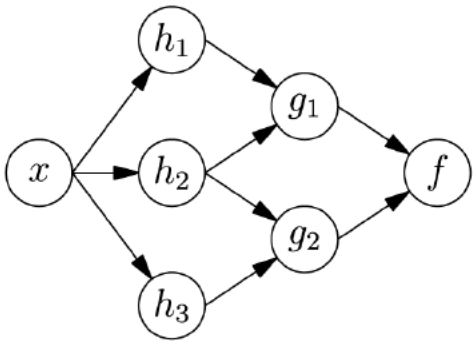
In general, the following "ingredients" are required to do deep learning.

- A suitable dataset
- A suitable loss function (e.g. Cross-Entropy)
- An algorithm for computing the gradient of the loss function (e.g. Backpropagation)
- An algorithm for updating the weights of the network (e.g. ADAM)
- A metric for estimating the model's performance (e.g. Accuracy)

B. The computational graph:

$$o = a(\mathbf{w} \cdot \mathbf{x} + b)$$

$$o = f(x) = f(g_1(h_1(x), h_2(x)), g_2(h_2(x), h_3(x)))$$



A. Bird classification task (2 Marks)

- 0.5 mark: Correctly identifies it as a *supervised image classification* task.
- 0.5 mark: Mentions dataset collection & labeling (including splitting into train/val/test).
- 0.5 mark: Includes preprocessing/augmentation or feature preparation.
- 0.5 mark: Mentions model training & evaluation → (loss function, metrics, deployment).

  (Full 2 marks for a clear end-to-end pipeline; partial if steps are missing or vague.)

B. Computational graph (3 Marks)

Equation 1: o=a(w·x+b) (1 mark)

- 0.5 mark: Shows weighted sum of inputs and bias node.
- 0.5 mark: Shows activation function node producing output.

Equation 2: o=f(g1(h1(x),h2(x)), g2(h2(x),h3(x))) (2 marks)

- 0.5 mark: Shows input x branching into h1,h2,h3.
- 0.5 mark: Correctly connects h1,h2→g1.
- 0.5 mark: Correctly connects h2,h3→g2.
- 0.5 mark: Combines g1,g2 into final function f→o.

(Full 3 marks if both graphs are complete and the flow is correct. Deduct marks for missing nodes or wrong flow.)

Q.3. You are given a CNN architecture defined in PyTorch as follows: **[10 Marks]**

```
nn.Sequential(
    nn.Conv2d(1, 5, kernel_size=3, stride=1, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),
    nn.Conv2d(5, 10, kernel_size=5, stride=1, padding=0),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),
    nn.Conv2d(10, 20, kernel_size=2, stride=2, padding=0),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=5, stride=2),
    nn.Flatten(),
    nn.Linear(2800, 6)
)
```

Assume that the input is a grayscale image of size 192 × 256 × 1. Now, answer the following questions:

A. Compute the output size (height × width × depth) after each (Conv and Pooling) layer (3.5M)
B. Calculate the number of trainable weights and biases in each convolutional layer (1.5M)
C. Verify that the flattened input to the fully connected layer matches the expected dimension (1M)
D. Compute the number of trainable parameters in the fully connected (Linear) layer. (1M)
E. Briefly explain in 1-2 sentences why a CNN is an adequate choice of architecture for image classification (1M)
F. Why is ReLU (and its variants) often preferred over sigmoid/tanh in modern CNNs? Provide two clear reasons (1-2 sentences). (1M)
G. If we add Batch Normalization after Conv1 and Conv3 and Dropout (p=0.5) before FC, explain the role of each and how they impact training/generalization. (1M)

Answer Key

A) Sizes after each Conv/Pool (3.5M)

1. Conv1 (1→5, k3,s1,p1): 192×256×5
2. MaxPool1 (k2,s2): 96×128×5
3. Conv2 (5→10, k5,s1,p0): 92×124×10
4. MaxPool2 (k2,s2): 46×62×10
5. Conv3 (10→20, k2,s2,p0): 23×31×20
6. MaxPool3 (k5,s2): 10×14×20

B) Trainable weights & biases per Conv (1.5M)

- Conv1: weights =5×(1·3·3)=45, biases =5→ 50
- Conv2: weights =10×(5·5·5)=1250, biases =10 → 1260
- Conv3: weights =20×(10·2·2)=800, biases =20→ 820

C) Flatten check (1M)

After the last pool: 10·14·20=∗∗2800∗∗ → matches Linear input 2800.

D) FC parameters (1M)

Linear(2800→6): weights =2800×6=16800, biases =6 → 16,806.

E) Why CNN for images? (1M)

CNNs exploit local receptive fields and weight sharing, capturing spatial hierarchies and translation-invariant features, ideal for images.

F) Why ReLU over sigmoid/tanh? (1M)

ReLU is non-saturating for x>0 (mitigates vanishing gradients) and is computationally cheap while inducing sparse activations, improving optimization and generalization.

G) Effect of BatchNorm & Dropout (1M)

BatchNorm (after Conv1/Conv3) normalizes activations, stabilizes/accelerates training, and permits higher learning rates.

Dropout (p=0.5) before FC randomly zeros units during training to reduce co-adaptation/overfitting, improving generalization (disabled at inference).

Q.4. **I)** You are given a simple RNN encoder with the following configuration: **[10 Marks]**

- Vocabulary embeddings (each word mapped to a 2-dimensional vector):

$$x_{\text{Deep}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix},\ x_{\text{learning}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix},\ x_{\text{works}} = \begin{bmatrix} -1 \\ -1 \end{bmatrix},\ x_{\langle stop \rangle} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

- Input-to-hidden weight matrix:

$$W_{hx} = \begin{bmatrix} 0 & -1 \\ 2 & 0 \\ 1 & 1 \end{bmatrix}$$

- Recurrent weight matrix: $W_{hh}=I_{(identity)}$
- Initial hidden state: $h_0=0$
- Biases: all zero.
- Activation: ReLU applied elementwise.

Compute the final hidden state h_4 of the encoder for the input sequence "Deep learning works <stop>"

Show your step-by-step calculations, including:

A. The weighted sum at each time step. (2M)
B. The hidden state after applying ReLU at each time step. (3M)
C. The final hidden state h_4. (2M)

II) Imagine you are building a model to predict the sentiment of a movie review. The review is long (200+ words), but the key sentiment is expressed only near the end.

A. Explain in 1–2 sentences why a vanilla RNN would struggle with this task and how it impacts training. (1.5M)
B. In 1–2 sentences, describe which gating mechanism in an LSTM helps solve this issue and how it preserves the critical information. (1.5M)

Answer Key

I)

A) Weighted sums a_t (2M)

- $t = 1$ (Deep): $a_1 = W_{hx}[0, 1]^T + \mathbf{0} = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$
- $t = 2$ (learning): $a_2 = W_{hx}[1, 0]^T + h_1 = \begin{bmatrix} 0 \\ 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \\ 2 \end{bmatrix}$
- $t = 3$ (works): $a_3 = W_{hx}[-1, -1]^T + h_2 = \begin{bmatrix} 1 \\ -2 \\ -2 \end{bmatrix} + \begin{bmatrix} 0 \\ 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$
- $t = 4$ (stop): $a_4 = W_{hx}[1, 1]^T + h_3 = \begin{bmatrix} -1 \\ 2 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \\ 2 \end{bmatrix}$

B) Hidden states $h_t = \mathrm{ReLU}(a_t)$ (3M)

- $h_1 = \mathrm{ReLU}([-1, 0, 1]^T) = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$
- $h_2 = \mathrm{ReLU}([0, 2, 2]^T) = \begin{bmatrix} 0 \\ 2 \\ 2 \end{bmatrix}$
- $h_3 = \mathrm{ReLU}([1, 0, 0]^T) = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$
- $h_4 = \mathrm{ReLU}([0, 2, 2]^T) = \begin{bmatrix} 0 \\ 2 \\ 2 \end{bmatrix}$

C) Final hidden state (2M)

$$\boxed{h_4 = \begin{bmatrix} 0 \\ 2 \\ 2 \end{bmatrix}}$$

II)

A) vanilla RNN struggles with long sequences because of the vanishing/exploding gradient problem, making it hard to carry information from early timesteps to later ones. As a result, it fails to capture dependencies when key sentiment appears near the end of a long review.

B) The cell state and gating mechanisms (especially the forget and input gates) in an LSTM allow the network to preserve or update important information across long time steps, ensuring that critical sentiment near the end is remembered and influences the final prediction.

Q.5. Consider a sequence of three tokens with embeddings: [5 Marks]

X_1=[1,0], X_2=[0,1], X_3=[1,1]

Assume query (Q), key (K), and value (V) matrices are the identity matrix. Answer the following:

A. Compute the raw attention scores between X_1 and all tokens (including itself). (2M)
B. Apply the softmax function to normalize these scores. (2M)
C. In one sentence, explain what self-attention achieves in this example. (1M)

Answer Key

A. Raw attention scores from X1 (2M)

$$[s_{11}, s_{12}, s_{13}] = [\, X_1 \cdot X_1,\ X_1 \cdot X_2,\ X_1 \cdot X_3 \,] = [1,\ 0,\ 1].$$

B. Softmax over these scores (2M)

$$\text{softmax}([1,0,1]) = \frac{[e^1, e^0, e^1]}{e^1 + e^0 + e^1} = \left[\frac{e}{2e+1}, \frac{1}{2e+1}, \frac{e}{2e+1}\right] \approx [0.422,\ 0.155,\ 0.422].$$

C. One-sentence explanation (1M)

Self-attention lets token X1 compute a **content-weighted mixture** of tokens, giving high weight to tokens most similar to it (here X1 and X3) while down-weighting unrelated ones (X2).

Q.6. **I)** Assume you are working for a renewable energy company that wants to forecast the next 24 hours of electricity demand using hourly data (features: temperature, past demand, and weather indicators). **[3 Marks]**

A. Propose a deep learning model (e.g., LSTM, CNN-LSTM, GRU) suitable for this task.

B. Briefly justify how the model can handle multivariate inputs, temporal dependencies, and noisy data. *(Limit answer to 1–2 sentences*

II) Your team is exploring Neural Architecture Search (NAS) to design better models, but faces high computational costs. **[2 Marks]**

A. Explain, in 1–2 sentences, how one-shot learning helps reduce this cost.

B. Mention one trade-off or challenge when using this approach.

Answer Key

I) Deep learning model of choice (1) and Justification (2M)

II) A. Cost reduction (1M):

One-shot NAS trains a supernet once and reuses its weights to evaluate many candidate architectures, avoiding retraining each from scratch.

B. Trade-off (1M):

The shared weights may give biased or inaccurate performance estimates, so the best architecture found might not be truly optimal.